

Digital Design with HDL

Lecture 5

Electronic and Communication Engineering
Department

Digital Systems Design Lab

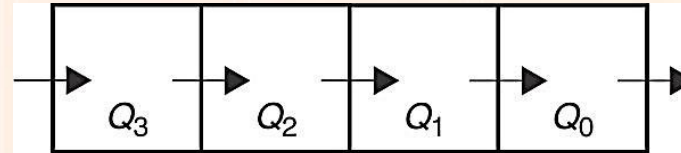
Salam Kadhim

salam.alshemmari@uokufa.edu.iq

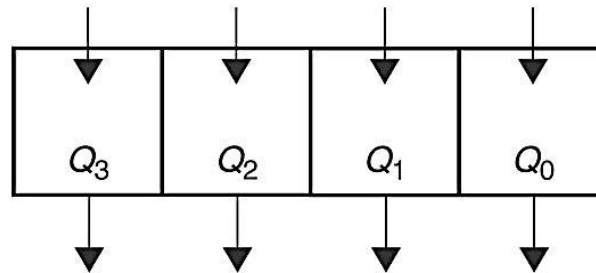
Shift Registers

- **Right Shift:** A movement of data from left to right in the shift register (toward the LSB). One bit shift per clock pulse.
- **Left Shift:** A movement of data from right to left in the shift register (toward the MSB). One bit shift per clock pulse.
- **Rotation:** Serial shifting (right or left) with the output of the last FF connected to the input of the first. Results in continuous circulation of SR data.

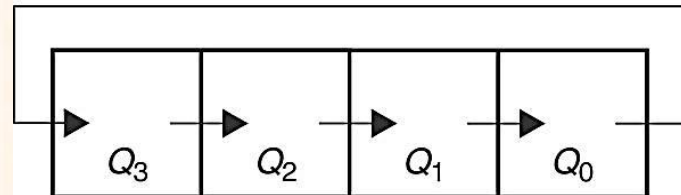
Shift Registers



a. Serial shifting

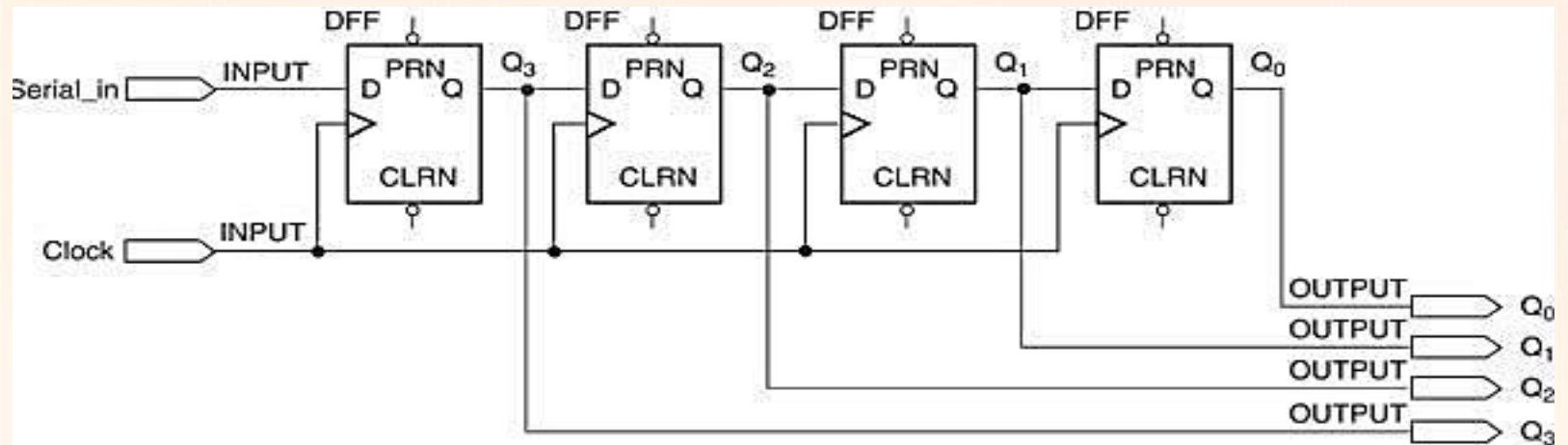


b. Parallel transfer



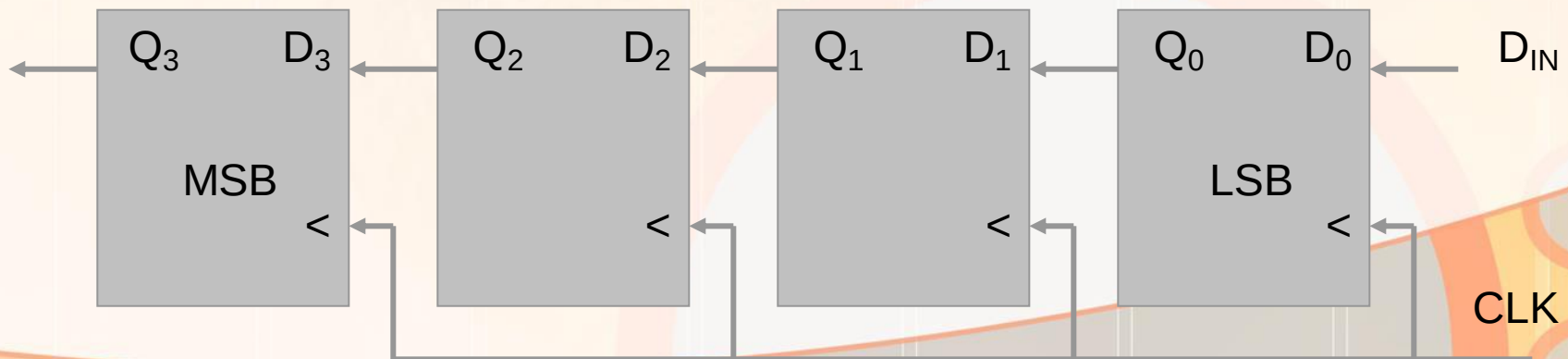
c. Rotation

Shift Registers

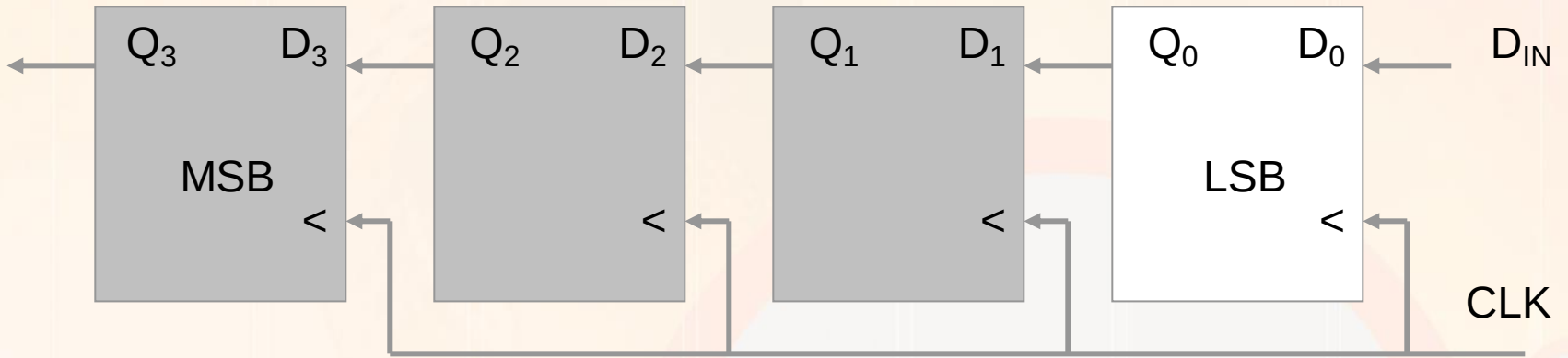


Serial Shift Register (SR)

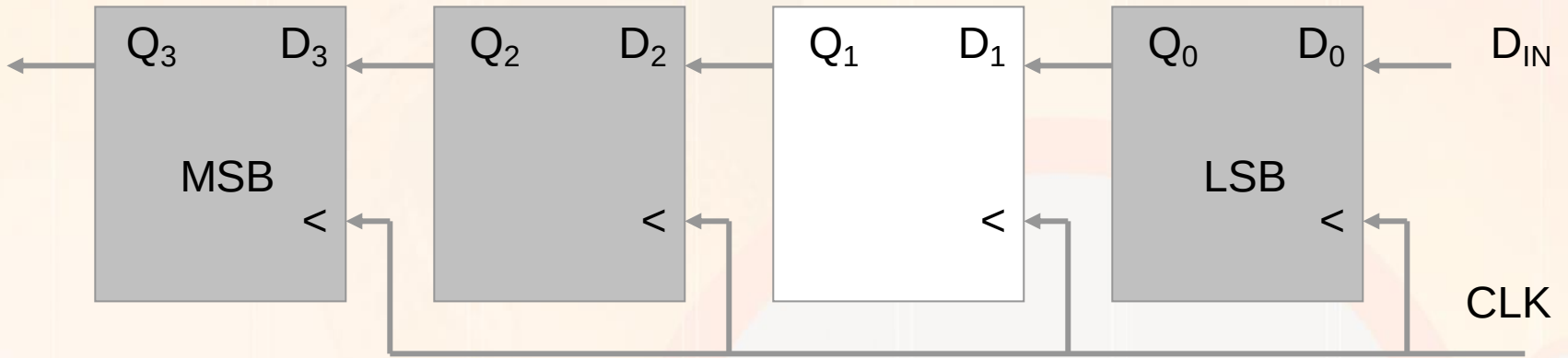
- *A 4-Bit Left Shift Register.*
- *D_{IN} is shifted into the LSB FF and shifted toward the MSB.*



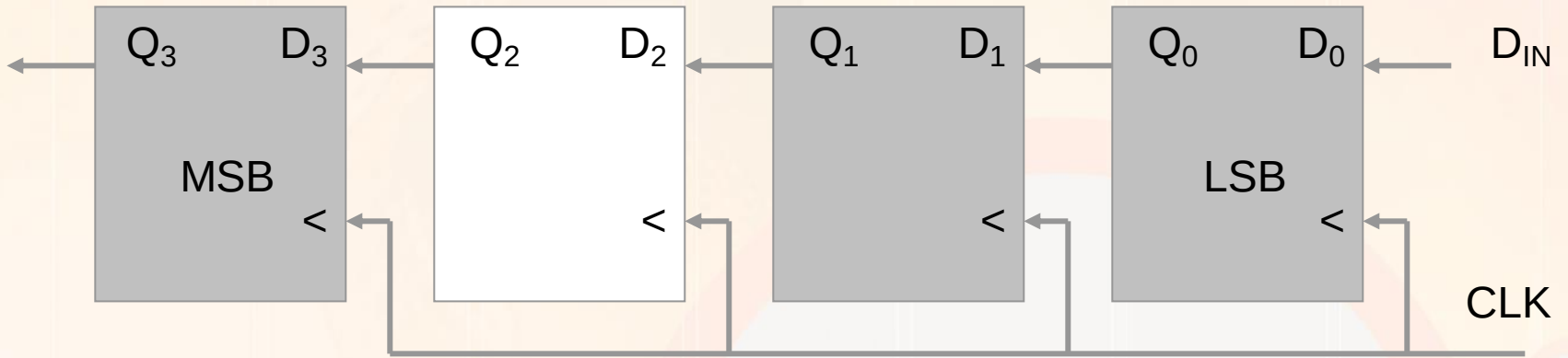
SS Left Shift – 1



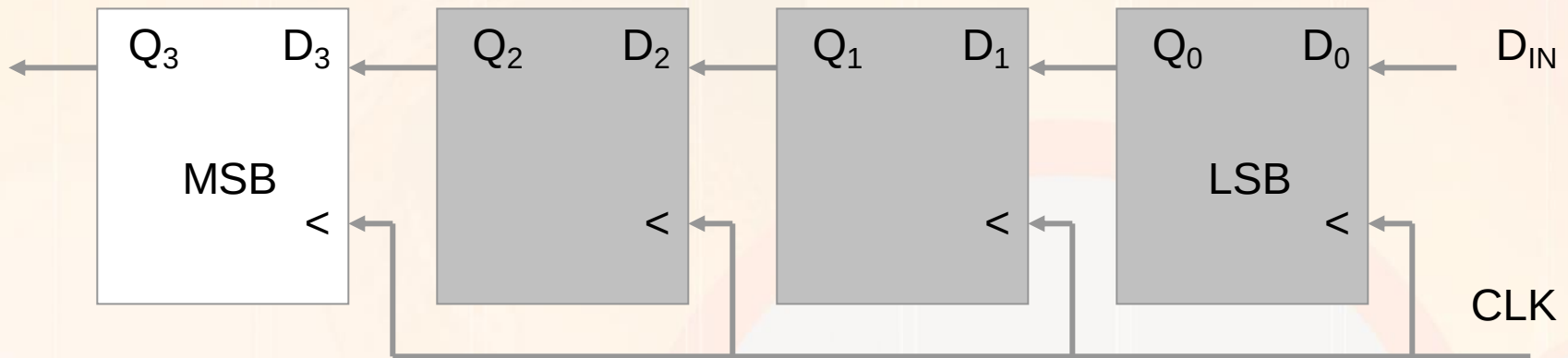
SS Left Shift – 2



SS Left Shift – 3



SS Left Shift – 4



Bi-Directional Shift Register

- When $DIR = 0$, the path of `Left_Shift_In` is selected.

$$Q_3 \Leftarrow Q_2 \Leftarrow Q_1 \Leftarrow Q_0 \Leftarrow$$

- *When $DIR = 1$, it selects the Right Shift In Path.*

$$Q_3 \Rightarrow Q_2 \Rightarrow Q_1 \Rightarrow Q_0 \Rightarrow$$

Direct Instantiation

```
library ieee;
use ieee.std_logic_1164.all;
-----
entity top is
    port(a: in std_logic;
         b: in std_logic;
         c: in std_logic;
         d: out std_logic);
end top;
-----
architecture structural of top is
    signal temp: std_logic;
begin
    or1: entity work.or_gate
        port map (a => a, b => b, c => temp);
    or2: entity work.or_gate
        port map (a => temp, b => c, c => d);
end structural;
```

No component declaration:
relies on presence of **work** library

Component instantiation:
creates an **instance** of the component, identified by a **unique label**. Must specify that the component is to be found within the **work** library.

PREFERRED

A word of caution...

- *Some texts (and many websites) propose an alternate notation for port maps, which does not involve the explicit definition of I/Os*

```
architecture structural of top is
    signal temp: std_logic;
begin
    or1: entity work.or_gate
        port map (a, b, temp);
    or2: entity work.or_gate
        port map (temp, c, d);
end structural;
```

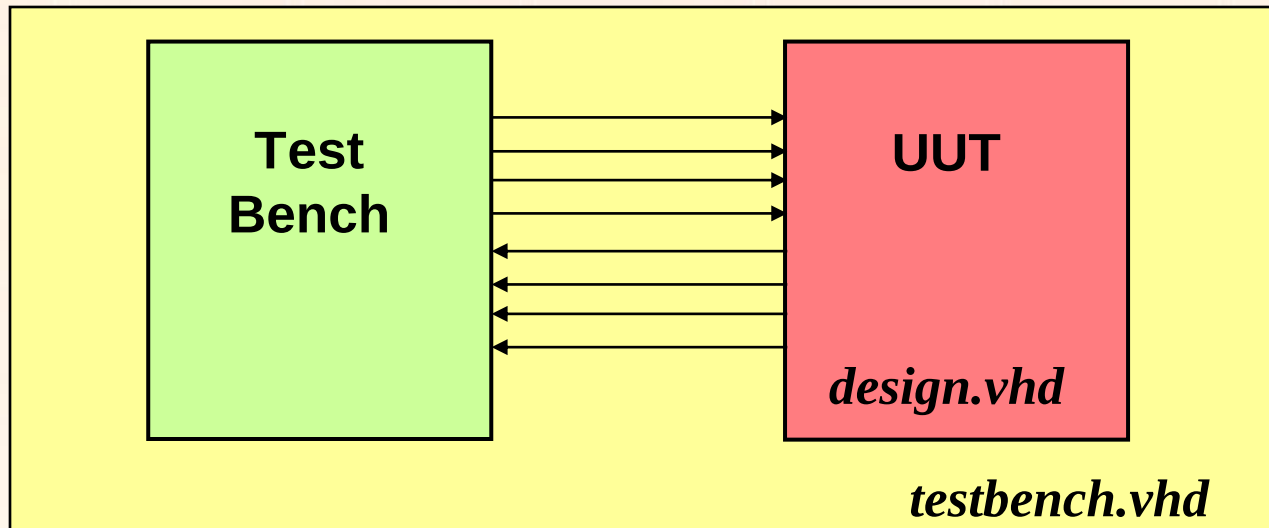
The order of the I/Os must be the same as in the component declaration

BAD

- *This is NOT a good idea (I/Os can be changed as the circuit is designed and tested) and should be avoided (particularly in direct instantiation)*

VHDL testbenches

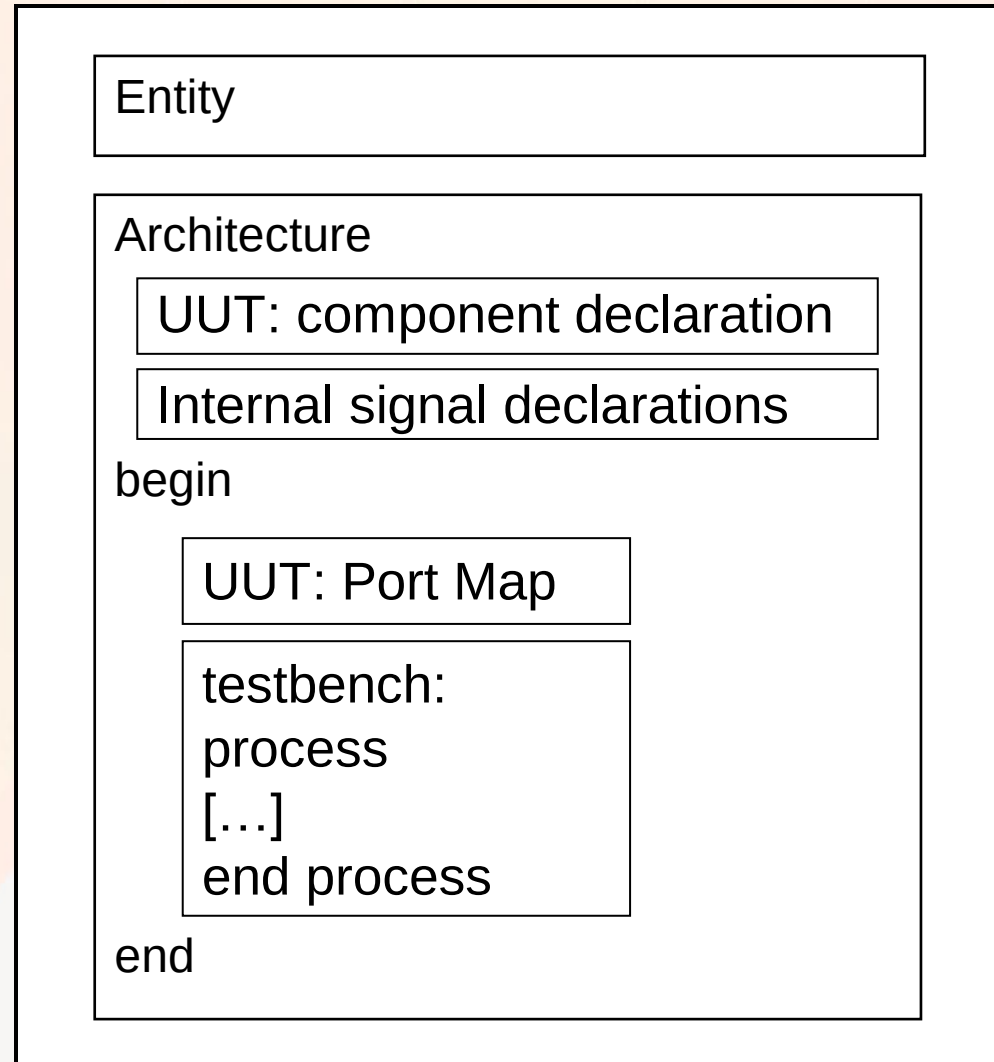
- A *VHDL testbench* is a VHDL module used to verify the logic and timing of a design (the *UUT* or Unit Under Test) by defining *input stimuli* and (optionally) *expected outputs*.



- In other words, they implement the *test strategy* for the design: they are usually applied to all components before they are integrated in a full design (hierarchical design).
- They are often very complex entities!

VHDL testbenches

- A VHDL testbench consists of the Unit Under Test and a series of input stimuli and output checks.
- A VHDL test bench is *not synthesisable* – it is not to be made into a circuit
- The Unit Under Test is included as a *component* in the testbench



VHDL testbenches

- You will be asked to select the VHDL entity to be tested (the UUT).*

```
entity full_adder is
    port (A,B,Cin: in std_logic;
          S, Cout: out std_logic);
end full_adder;

architecture arch of full_adder is
begin
    Cout <= Cout <= ((A xor B) and Cin) or (A and B);
    S <= (A xor B) xor Cin;
end arch;
```

- The tools will generate a "skeleton" testbench:*
 - First, an **entity** with no inputs or outputs*

```
entity test_dut_vhd is
end test_dut_vhd;
```

VHDL testbenches

- A "skeleton" testbench:
 - Then, an *architecture* declaration that includes the *UUT* as a component and *signals* connected to its inputs and outputs

```
architecture behavior of test_dut_vhd is
  -- component declaration for the unit under test (uut)
  component full_adder
    port( a : in  std_logic;
          b : in  std_logic;
          cin : in std_logic;
          s : out std_logic;
          cout : out std_logic);
  end component;
  --inputs
  signal a, b, cin : std_logic := '0';
  --outputs
  signal s, cout : std_logic := '0';
```

Initial value : to be avoided
for synthesizable VHDL!!!!
(but testbenches are NOT synthesizable!)

VHDL testbenches

- A "skeleton" testbench:
 - The UUT component is *instantiated*, the signals are connected and *process* is created.

```
begin
  -- instantiate the unit under test (uut)
  uut: full_adder port map(
    a => a;
    b => b;
    cin => cin;
    s => s;
    cout => cout);

  tb : process
  begin
    -- wait 100 ns for global reset to finish
    wait for 100 ns;
    -- place stimulus here
    wait; -- will wait forever
  end process;
end;
```

The "wait for" statement is NOT SYNTHESIZABLE!!! – but testbenches are not meant to be synthesized.

This delay represents the initial internal FPGA reset – DO NOT REMOVE!!!

VHDL testbenches

- The *stimuli* (inputs) to test your circuit will have to be instantiated within the process in the form of *timed assignments to the inputs of the UUT*

```
tb : process
begin
    -- wait 100 ns for global reset to finish
    wait for 100 ns;
    -- place stimulus here
    a <= '0';
    b <= '0';
    cin <= '0';
    wait for 10ns;
    cin <= '1';
    wait for 10ns;
    cin <= '0';
    b <= '1';
    [...]
    wait; -- will wait forever
end process;
```

Wait for an (almost) arbitrary amount of time (for combinational circuits) – you still want to make it “reasonable”

VHDL testbenches

- For sequential circuits, a separate *clock process* after the UUT declaration is used to generate the clock with an *infinite loop*. The ALTEA wizard will generate this for you

```
[...]
constant clk_period : time := 100ns;
BEGIN
    -- Instantiate the Unit Under Test (UUT)
    uut: my_design PORT MAP( [...], clk=>clk, [...] );

    process -- clock process for CLK
    begin
        CLOCK_LOOP : loop
            clk <= '0';
            wait for clk_period/2;
            clk<= '1';
            wait for clk_period/2;
        end loop CLOCK_LOOP;
    end process;

    tb : process
        wait for 100ns - reset
        [...]
        wait for clk_period*10;
        [...]
```

Clock period defined
by constant

"loop", "wait for"
are non-synthesizable

Clock signal automatically
extracted from design by wizard

Use of a constant allows clock
frequency to be changed

Delays between input test
patterns should always
defined as multiples (or
fractions) of the clock period

Synthesizable VHDL

- VHDL looks like a programming language
 - Its syntax is derived from Ada, a high-reliability software language
- But it is **NOT** a programming language!!!!
- Writing good VHDL requires a different approach to writing good software
- This is particularly true for **synthesizable** VHDL, where the designer should always consider that everything they write will have to be converted into hardware
- There is an exception to this rule: **testbenches** will never be synthesized, and their code is actually executed (in simulations)

VHDL testbenches

- *Write a testbench to verify the operation of the 16-bit shift register with load*

Bidirectional 16-bit shift register with load

```
entity SREG_16BIT is
  port (CLK, LD, SH, DIR, RST: in STD_LOGIC;
        Data: in STD_LOGIC_VECTOR(15 downto 0);
        Q : out STD_LOGIC_VECTOR(15 downto 0));
end SREG_16BIT;

architecture arch of SREG_16BIT is
  signal Qint : STD_LOGIC_VECTOR(15 downto 0);
begin
  SREG16: process
  begin
    if (rising_edge(CLK)) then
      if (RST = '1') then
        Qint <= (others => '0');
      elsif (LD = '1') then
        Qint <= Data;
      elsif (SH = '1') then
        if (DIR = '1') then
          Qint(15 downto 1) <= Qint(14 downto 0);
          Qint(0) <= '0';
        else
          Qint(14 downto 0) <= Qint(15 downto 1);
          Qint(15) <= '0';
        end if;
      end if;
    end if;
  end process SREG16;
  Q <= Qint;
end arch;
```